

Módulo 6 – Capítulo 08 – Sección 01

RAG multimodal: imágenes, audio y documentos con tablas y diagramas

El conocimiento de una organización raramente existe exclusivamente en forma de texto plano. Un manual técnico de mantenimiento de aeronaves tiene diagramas de los sistemas hidráulicos, tablas de tolerancias de componentes y fotografías de los procedimientos. Un reporte financiero trimestral tiene gráficos de tendencias, tablas de resultados por segmento y notas al pie con contexto regulatorio. Una sesión de entrenamiento en vídeo tiene la voz del instructor, capturas de pantalla de la interfaz y subtítulos de texto. Si el sistema RAG solo indexa el texto plano extraíble de estos documentos, está accediendo a una fracción del conocimiento real que contienen; en muchos corpus empresariales, entre el 30% y el 60% del conocimiento relevante está codificado en formatos no textuales que un pipeline RAG estándar ignora. RAG multimodal extiende el paradigma para incluir estos formatos como fuentes de conocimiento recuperables.

La arquitectura general del RAG multimodal tiene dos enfoques principales con distintos trade-offs de complejidad, calidad y costo. El **enfoque caption-based** convierte cada elemento no textual en texto mediante modelos de visión o herramientas de extracción especializadas, y luego indexa ese texto con embeddings estándar. Una tabla de un contrato se convierte en Markdown con separadores `|`; un diagrama de flujo se convierte en una descripción textual de sus nodos y relaciones generada por GPT-4V o Claude Sonnet; un gráfico de barras se convierte en "un gráfico de barras que muestra las ventas trimestrales del Q1 al Q4 de 2024, con Q3 como pico de \$2.3M y Q1 como mínimo de \$1.1M". Este texto convertido se indexa como cualquier otro chunk de texto, con la ventaja de que puede recuperarse con el mismo pipeline de búsqueda semántica existente sin cambios en la infraestructura. La desventaja es que la calidad de la recuperación depende de la calidad de la conversión: una caption imprecisa produce un vector incorrecto que no será recuperado para las queries relevantes.

El **enfoque native multimodal embedding** usa modelos de embedding que pueden generar vectores en el mismo espacio latente para texto e imágenes, permitiendo búsqueda

cross-modal: una query en texto puede recuperar directamente imágenes relevantes sin necesidad de que las imágenes tengan captions textuales. CLIP (Contrastive Language-Image Pre-Training, OpenAI), SigLIP (Sigmoid Loss for Language Image Pre-Training, Google) y voyage-multimodal-3 (Voyage AI) son los modelos más representativos de este enfoque. Un vector generado por CLIP para la query "diagrama de instalación del motor" puede recuperar directamente imágenes de diagramas de instalación del corpus aunque no tengan ninguna descripción textual asociada. Este enfoque es conceptualmente más elegante pero requiere infraestructura de embedding más compleja y modelos que soporten ambas modalidades.

Las **herramientas de extracción de contenido no textual** son el componente que más frecuentemente determina la calidad del RAG multimodal. **Docling** de IBM es actualmente la herramienta open source más robusta para documentos complejos: usa modelos de comprensión de layout visual para detectar columnas, tablas, figuras y su jerarquía, reconstruye el orden de lectura correcto incluso en layouts complejos de múltiples columnas, y extrae tablas en formato estructurado con alta fidelidad (>90% de accuracy para tablas con bordes explícitos). **Azure Document Intelligence** (anteriormente Form Recognizer) ofrece capacidades similares como servicio managed con soporte para formularios preentrenados de tipos de documentos comunes (facturas, recibos, contratos). **Camelot** y **pdfplumber** son opciones ligeras para extracción de tablas de PDFs con bordes explícitos, con precisión más baja para layouts complejos pero sin la dependencia de un servicio externo.

El **procesamiento de audio** sigue el patrón caption-based de forma natural: **Whisper** de OpenAI (disponible en varios tamaños desde tiny hasta large-v3) transcribe audio a texto con accuracy superior al 95% para inglés y soporte de 99 idiomas, con timestamps por segmento que permiten recuperar el segmento específico del audio que contiene la información relevante. Las transcripciones se indexan como texto estándar, con un metadato de timestamp que el sistema puede usar para generar un enlace al segmento exacto del vídeo o audio como fuente citada en la respuesta del LLM.

Técnicas y herramientas para RAG multimodal

- **Extracción de tablas de PDFs:** Docling (IBM, open source) para layouts complejos; Camelot lattice para tablas con bordes (>95% accuracy); Camelot stream para tablas sin bordes (70–85%); Azure Document Intelligence para formularios y tablas en documentos escaneados; representar en Markdown con | separadores para indexación textual.
- **Caption generation para imágenes:** GPT-4o o Claude Sonnet para generación de captions descriptivas de 100–200 palabras; incluir en el caption: tipo de imagen,

elementos principales, datos numéricos visibles, pregunta implícita que responde la imagen; la calidad del caption determina directamente el recall de recuperación de la imagen.

- **CLIP/SigLIP para búsqueda cross-modal:** vectores en el mismo espacio latente para texto e imágenes; búsqueda semántica directa de imágenes desde queries de texto; ideal para corpus con imágenes que no tienen captions naturales; requiere base vectorial que soporte almacenamiento de embeddings de imagen.
- **Whisper para transcripción de audio:** modelos desde tiny (39M params) hasta large-v3 (1550M params); accuracy >95% en inglés, >85% en la mayoría de idiomas europeos; timestamps por segmento para recuperación precisa; disponible como API (openai.audio.transcriptions) o self-hosted.
- **Layout analysis para documentos complejos:** Docling y Azure Document Intelligence detectan estructura espacial (columnas, encabezados, footers) y reconstruyen el orden de lectura lógico; crítico para PDFs de reportes financieros, artículos científicos y contratos con layouts no lineales.
- **Multimodal context para el LLM generador:** enviar imágenes recuperadas como base64 a modelos con visión (GPT-4o, Claude Sonnet); enviar tablas como Markdown a modelos de solo texto; los modelos con visión pueden razonar directamente sobre tablas e imágenes en el contexto sin conversión previa.

El RAG multimodal amplía dramáticamente el alcance del sistema, pero añade complejidad significativa en la etapa de ingesta. La regla práctica es empezar por los formatos más frecuentes en el corpus específico: en la mayoría de los corpus empresariales, las tablas de PDFs y las imágenes captionadas cubren el 80% del conocimiento no textual con el 20% del esfuerzo de implementación. La siguiente sección aborda la extensión al otro gran dominio de conocimiento empresarial no textual: los datos estructurados en bases de datos relacionales.

Módulo 6 – Capítulo 08 – Sección 02

RAG sobre datos estructurados: Text-to-SQL y NL2API

Hay una clase de preguntas que los usuarios de un sistema de asistencia empresarial hacen con frecuencia y que el RAG sobre documentos de texto no puede responder: "¿cuáles son los 10 clientes con mayor facturación en el último trimestre?", "¿qué empleados tienen más de 5 años de antigüedad y están en el departamento de ventas?", "¿cuántos tickets de soporte se abrieron esta semana con prioridad alta?". La respuesta a estas preguntas no está en documentos indexados sino en bases de datos relacionales, data warehouses o APIs con esquemas definidos. RAG sobre datos estructurados —implementado principalmente como Text-to-SQL o NL2API— extiende el sistema para responder estas preguntas accediendo a los datos en tiempo real.

Text-to-SQL es el problema de traducir una pregunta en lenguaje natural a una query SQL correcta y ejecutable. Los LLMs modernos tienen capacidades Text-to-SQL sólidas cuando se les proporciona el contexto correcto: el esquema de la base de datos (nombres de tablas, columnas, tipos, claves foráneas, relaciones), algunas filas de ejemplo de las tablas más relevantes, y opcionalmente 3–5 ejemplos de pares (pregunta, SQL) del dominio específico (few-shot examples). GPT-4o y Claude 3.5 Sonnet alcanzan accuracy del 70–85% en el benchmark Spider (Text-to-SQL estándar) en esquemas de complejidad media con few-shot examples bien elegidos; la accuracy cae al 40–60% para esquemas con más de 50 tablas relacionadas donde el contexto de esquema completo no cabe en el prompt de forma útil.

El **patrón RAG híbrido para Text-to-SQL** resuelve el problema del esquema de base de datos grande mediante una primera etapa de recuperación semántica: antes de generar el SQL, el sistema indexa el esquema de la base de datos como texto (cada tabla y sus columnas como un documento, con descripciones de negocio de las columnas si están disponibles en el diccionario de datos) y usa búsqueda semántica para seleccionar las 5–10 tablas más relevantes para la query específica del usuario. Solo el esquema de esas tablas se incluye en el prompt de generación SQL, reduciendo el contexto de esquema de potencialmente cientos de tablas a las directamente relevantes. Esta estrategia de schema-aware retrieval mejora la accuracy del Text-to-SQL en 15–25 puntos para bases de datos de más de 20 tablas.

El ciclo de **SQL validation y error recovery** es crítico para la usabilidad en producción. El LLM comete errores en el SQL generado con frecuencia (tablas con nombres ligeramente distintos a los del esquema, joins incorrectos, filtros de fecha mal formateados). El pipeline debe: (1) intentar ejecutar el SQL generado en un entorno de solo lectura; (2) si el motor de base de datos devuelve un error, enviar el SQL fallido y el mensaje de error de vuelta al LLM para que genere una versión corregida; (3) repetir hasta 3 veces antes de responder al usuario que no pudo generar una query correcta. Este ciclo de auto-corrección reduce la tasa de errores finales en un 30–50% respecto a un sistema sin recuperación de errores.

La **seguridad en Text-to-SQL** es un requisito que no puede ignorarse. El LLM puede generar queries que el sistema ejecuta en la base de datos: si el usuario de base de datos del sistema tiene permisos de escritura, un LLM manipulado (por ejemplo, a través de prompt injection en la query del usuario) podría generar operaciones INSERT, UPDATE o DROP TABLE que modifiquen o destruyan datos de producción. Las medidas de mitigación son: crear un usuario de base de datos de solo lectura dedicado específicamente para el sistema Text-to-SQL, con acceso restringido a las tablas que el sistema necesita consultar; validar que el SQL generado no contenga operaciones DDL ni DML antes de ejecutarlo (un simple check de keywords prohibidas: DROP, DELETE, UPDATE, INSERT, ALTER, TRUNCATE); y monitorear y loggear todas las queries ejecutadas para auditoría.

NL2API aplica el mismo principio a APIs REST: en lugar de generar SQL, el LLM genera la llamada a una API correcta (endpoint URL, parámetros, headers de autenticación) a partir de la pregunta del usuario. La especificación OpenAPI/Swagger del API se usa como contexto para el LLM (los endpoints disponibles, sus parámetros y los tipos de respuesta). El framework ReAct y el mecanismo de function calling (OpenAI tools, Claude tool use, Gemini function declarations) son las implementaciones más comunes: el LLM declara qué función (herramienta) quiere llamar, el sistema la ejecuta y devuelve el resultado al LLM para que genere la respuesta final.

Técnicas de RAG sobre datos estructurados

- **Schema-aware retrieval:** indexar el esquema de la base de datos como documentos semánticos; buscar las 5–10 tablas más relevantes para la query antes de generar el SQL; reducir el contexto de esquema al subconjunto relevante; mejora accuracy en 15–25 puntos para bases de datos de >20 tablas.
- **Few-shot examples de SQL:** incluir 3–5 ejemplos de pares (pregunta, SQL) del dominio específico en el prompt de generación; elegir ejemplos que cubran los patrones

de query más frecuentes (JOINS, GROUP BY, filtros de fecha, subconsultas); diferencia de 10–20 puntos de accuracy en el dominio específico.

- **SQL validation y error recovery:** ejecutar el SQL en entorno de solo lectura → capturar error → enviar error + SQL original al LLM → iterar hasta 3 veces; reduce tasa de errores finales en 30–50%; implementable con un simple try/except que alimenta el error de vuelta al LLM.
- **Seguridad de solo lectura:** usuario de base de datos dedicado con permisos SELECT únicamente; validar que el SQL no contiene palabras clave prohibidas (DROP, DELETE, UPDATE, INSERT, ALTER) antes de ejecutar; loggear todas las queries para auditoría.
- **Caché de queries SQL:** cachear pares (query_normalizada, SQL_correcta) para preguntas frecuentes; similar al caching semántico para RAG textual; evita la generación SQL y ejecuta la query cacheada directamente; especialmente útil para dashboards o reportes recurrentes.
- **NL2API con function calling:** definir herramientas con nombre, descripción y esquema JSON de parámetros; el LLM emite tool_calls con los argumentos correctos; el sistema ejecuta la función real y devuelve el resultado al LLM; GPT-4o y Claude 3.5 Sonnet tienen function calling nativo de alta calidad.

Text-to-SQL es un componente de alta complejidad y riesgo que requiere diseño cuidadoso de la capa de seguridad antes de la implementación. Cuando se implementa correctamente, amplía dramáticamente las capacidades del sistema RAG para responder preguntas sobre datos en tiempo real que ningún sistema de recuperación de documentos puede responder. La siguiente sección explora GraphRAG, la extensión para corpus donde las relaciones entre entidades son parte central del conocimiento.

Módulo 6 – Capítulo 08 – Sección 03

GraphRAG: recuperación basada en grafos de conocimiento

Existe una clase de preguntas que el RAG vectorial estándar no puede responder bien, no por limitaciones del modelo de embedding ni del chunking, sino por limitaciones estructurales del paradigma de búsqueda por similitud de un único vector de consulta: las preguntas de síntesis global sobre un corpus. "¿Cuáles son los principales debates académicos sobre el uso de IA en diagnóstico médico en los últimos tres años?", "¿qué organizaciones están más mencionadas en relación con el cambio regulatorio del sector X?", o "¿cuáles son los temas más frecuentes en los tickets de soporte de clientes enterprise este trimestre?". Responder estas preguntas requiere agregar información de decenas o cientos de documentos relacionados, identificar patrones y relaciones entre entidades, y sintetizar una visión de conjunto. La búsqueda vectorial puntual recupera los documentos más similares a un vector de consulta, no los más representativos del conjunto de documentos relevantes para una pregunta de síntesis.

GraphRAG, publicado como sistema open source por Microsoft Research en 2024, propone una solución: en lugar de indexar el corpus como un conjunto plano de chunks de texto, construir un **grafo de conocimiento** donde los nodos representan entidades extraídas del corpus (personas, organizaciones, conceptos, eventos, regulaciones) y las aristas representan las relaciones entre ellas (pertenece_a, publicó, contradice, relacionado_con). Este grafo, construido automáticamente usando un LLM para extraer entidades y relaciones de cada chunk durante la ingesta, codifica el conocimiento del corpus en una estructura navegable que permite responder preguntas de síntesis recorriendo el grafo de relaciones.

El proceso de construcción del grafo tiene cuatro pasos. Primero, **extracción de entidades y relaciones**: para cada chunk del corpus, un LLM (GPT-4o-mini o Claude Haiku) extrae las entidades mencionadas y las relaciones entre ellas en formato JSON estructurado. El costo es aproximadamente \$0.01–0.05 por documento con GPT-4o-mini, lo que puede representar cientos o miles de dólares para corpus grandes. Segundo, **resolución de entidades**: unificar entidades que se refieren a la misma cosa pero se mencionan con nombres ligeramente distintos ("IBM", "International Business Machines", "IBM Corp" → nodo único con alias). Tercero, **detección de comunidades**: aplicar el **Leiden algorithm** (un

algoritmo de clustering de grafos optimizado para comunidades de alta modularidad) para identificar grupos de entidades fuertemente relacionadas entre sí; cada comunidad es un "tema" emergente del corpus. Cuarto, **generación de resúmenes de comunidad**: usar un LLM para generar un resumen de 100–500 palabras de cada comunidad, capturando el conocimiento agregado de todos los documentos sobre ese tema.

El **routing de queries en GraphRAG** distingue dos tipos de búsqueda. Las queries "locales" —preguntas sobre entidades específicas o sobre relaciones concretas ("¿qué documentos mencionan al Dr. García en el contexto de su investigación sobre proteínas beta?")— se benefician del RAG vectorial estándar porque son preguntas de recuperación puntual. Las queries "globales" —preguntas de síntesis sobre el corpus completo ("¿cuáles son los temas de investigación emergentes en el corpus de publicaciones de bioquímica del último año?")— se benefician del GraphRAG porque requieren agregar información de múltiples comunidades del grafo. Un sistema GraphRAG maduro implementa un clasificador que enruta cada query al mecanismo de recuperación apropiado, capturando las ventajas de ambos enfoques.

Las **limitaciones de GraphRAG** son igualmente importantes de entender para calibrar cuándo es la solución correcta. El costo de construcción del grafo es entre 10 y 50 veces mayor que la indexación vectorial estándar (múltiples llamadas LLM por documento). Las queries locales son entre 30–50% más lentas que en RAG vectorial estándar porque requieren navegación del grafo además de la búsqueda vectorial. Y la calidad de las respuestas de síntesis depende de la calidad de la extracción de entidades y relaciones durante la ingesta: entidades mal extraídas o relaciones incorrectas contaminan el grafo y producen síntesis incorrectas.

Componentes técnicos de GraphRAG

- **Extracción del grafo de conocimiento**: LLM (GPT-4o-mini o Claude Haiku) extrae entidades y relaciones de cada chunk en JSON estructurado; costo \$0.01–0.05 por documento; el pipeline de extracción es la etapa de mayor costo en GraphRAG.
- **Community detection con Leiden algorithm**: algoritmo de clustering de grafos que maximiza la modularidad de las comunidades; genera comunidades temáticas coherentes del corpus; disponible en la librería `leidenalg` de Python.
- **Query routing local vs. global**: clasificar la query como local (información sobre entidades específicas) o global (síntesis del corpus completo); enrutar al RAG vectorial o al

GraphRAG según la clasificación; el router puede ser un LLM con prompt de clasificación o un clasificador más ligero.

- **Neo4j y bases de datos de grafos:** Neo4j (Cypher query language) o Amazon Neptune (TinkerPop/Gremlin) para almacenamiento del grafo; traversals del grafo en tiempo de query para encontrar entidades relacionadas; latencia de traversal 10–100ms para grafos de millones de nodos.
- **Microsoft GraphRAG pipeline:** herramienta open source en github.com/microsoft/graphrag que automatiza la extracción, clustering, generación de resúmenes y serving de queries; requiere configuración de LLM y almacenamiento; documentación activa y comunidad creciente.
- **GraphRAG vs. RAG vectorial:** GraphRAG supera al vectorial en queries de síntesis global (+26% en relevancia según el paper de Microsoft 2024); 2–5x más costoso de construir; 30–50% más lento en queries locales; el patrón hybrid que usa ambos según el tipo de query captura las ventajas de cada uno.

Nota del Arquitecto: GraphRAG es una de las técnicas más sobreusadas en la comunidad RAG en 2024. El criterio correcto para adoptarlo es simple: ¿los usuarios hacen frecuentemente preguntas de síntesis global sobre el corpus que el RAG estándar no responde bien? Si la respuesta es sí —corpus de investigación académica, corpus de análisis competitivo, corpus de análisis regulatorio— GraphRAG justifica su costo. Si la mayoría de las queries son factuales específicas sobre documentos individuales, el overhead de GraphRAG no aporta valor y el RAG vectorial con buen chunking es superior en calidad y costo.

GraphRAG amplía el dominio de problemas que un sistema RAG puede abordar, especialmente para corpus donde el conocimiento está distribuido en forma de redes de entidades relacionadas. La siguiente sección explora otra extensión: la integración de APIs externas como fuentes de información dinámica en tiempo real.

Módulo 6 – Capítulo 08 – Sección 04

RAG con herramientas: integración de APIs externas en el ciclo de recuperación

El índice vectorial contiene el conocimiento del corpus histórico del sistema: documentos procesados hasta el momento de la última actualización del índice. Pero existen categorías de información que ningún índice estático puede proporcionar: el precio actual de una acción, el estado en tiempo real de un ticket de soporte en Jira, la previsión meteorológica para mañana, el estado de entrega de un pedido. Estas informaciones son dinámicas por naturaleza —cambian continuamente y es imposible indexarlas de antemano— y requieren consulta directa a sistemas externos en tiempo real. El RAG con herramientas extiende el paradigma conectando el ciclo de respuesta del LLM con APIs externas que proveen información en tiempo real, transformando el sistema RAG de un sistema de consulta sobre conocimiento pasado a un sistema de acceso a información del presente.

El mecanismo fundamental es el **function calling** (también llamado tool use en la API de Anthropic o function declarations en la API de Gemini): una capacidad de los LLMs modernos que les permite emitir llamadas estructuradas a funciones externas en lugar de (o además de) generar texto de respuesta. El sistema define las herramientas disponibles con un nombre, una descripción en lenguaje natural de qué hace la herramienta y para qué es útil, y un esquema JSON de los parámetros que acepta. El LLM lee estas definiciones en el prompt del sistema y, cuando determina que necesita información de una herramienta para responder la query del usuario, emite un objeto JSON con el nombre de la herramienta y los valores de los parámetros. El sistema ejecuta la herramienta con esos parámetros, obtiene el resultado, y lo devuelve al LLM como un `tool_result` en el contexto de la conversación. El LLM entonces genera la respuesta final usando tanto los chunks del RAG vectorial como los resultados de las herramientas.

La **composición del conjunto de herramientas** es una decisión de diseño que determina las capacidades del sistema. En un asistente empresarial típico, el conjunto puede incluir: búsqueda vectorial en el índice interno (acceso al corpus documental), consulta a Salesforce CRM (estado actual de cuentas de clientes), consulta a Jira (tickets abiertos y su estado),

búsqueda en la web con Tavily o SerpAPI (información pública actualizada), y consulta a servicios internos de la empresa vía API REST. El LLM actúa como un router inteligente que selecciona qué herramienta usar según la naturaleza de la pregunta: "¿qué dice la política de devoluciones?" va al índice vectorial; "¿cuál es el estado del ticket SUPP-1234?" va a la API de Jira; "¿qué dijo el CEO de la empresa X en su última conferencia?" va a la búsqueda web.

El **parallel tool calling** es una optimización de latencia disponible en GPT-4o y Claude 3.5 Sonnet que permite al LLM solicitar múltiples herramientas simultáneamente en una sola inferencia. Para una query como "¿cuál es el precio actual de la acción X y qué dice el último informe anual sobre la estrategia de la compañía?", el LLM puede emitir simultáneamente una llamada a la API de precios de bolsa y una búsqueda vectorial en el índice de informes anuales. El sistema ejecuta ambas herramientas en paralelo, reduce la latencia total respecto a la ejecución secuencial, y proporciona ambos resultados al LLM en una sola respuesta.

La **gestión de errores de herramientas** es un componente que frecuentemente se omite en implementaciones iniciales con consecuencias dolorosas en producción. Las APIs externas fallan: timeouts, rate limits (la API de bolsa puede tener un límite de 5 llamadas por segundo), errores 503 por mantenimiento. El sistema debe implementar: timeouts máximos por herramienta (2–5 segundos) para no bloquear la respuesta al usuario indefinidamente; reintentos con backoff exponencial para errores transitorios; notificación al LLM cuando una herramienta falla, para que genere una respuesta parcial o solicite reformulación ("No pude obtener el precio actual de la acción X. ¿Quieres que responda sobre la información histórica disponible en el corpus?"); y logging de todos los errores para monitoreo.

Patrones de integración de APIs en RAG

- **Function calling para recuperación dinámica:** definir herramientas con nombre, descripción y esquema JSON; el LLM emite tool_calls con los argumentos; ejecutar la función real y devolver el resultado como tool_result; el LLM genera la respuesta final integrando todos los resultados.
- **Tool selection como retrieval routing:** el conjunto de herramientas incluye tanto búsqueda vectorial como APIs externas; el LLM selecciona qué herramienta usar según la naturaleza de la query; sistema natural de routing sin clasificador explícito.
- **Parallel tool calling:** GPT-4o y Claude 3.5 Sonnet soportan múltiples tool_calls en una sola inferencia; ejecutar en paralelo reduce latencia; especialmente útil para queries que requieren información de múltiples fuentes simultáneas.

- **Gestión de errores:** timeout máximo por herramienta (2–5 segundos); reintentos con backoff para errores transitorios; notificación al LLM de fallos para respuestas parciales informadas; logging de todos los errores para monitoreo.
- **Security boundary para tool execution:** sandbox con permisos mínimos para ejecutar herramientas; validar argumentos generados por el LLM antes de ejecutar (prevención de prompt injection en tool_calls); loggear todas las ejecuciones para auditoría.
- **Timeout y SLO por herramienta:** definir SLO de latencia por herramienta (búsqueda vectorial: 50ms, API de Jira: 500ms, búsqueda web: 2s); si la herramienta no responde en el tiempo límite, proceder sin su resultado notificando al LLM.

El RAG con herramientas amplía el alcance del sistema a información en tiempo real y a sistemas externos, pero introduce dependencias de APIs externas con sus propios SLAs, rates limits y fallos. Cada herramienta adicional es una dependencia a gestionar y un modo de fallo a diseñar. La siguiente sección aborda los desafíos técnicos específicos del corpus empresarial real: documentos con layouts complejos, idiomas mixtos y formatos técnicos especializados.

Módulo 6 – Capítulo 08 – Sección 05

Desafíos específicos: PDFs con layout complejo, idiomas y documentos técnicos

El corpus de producción raramente está compuesto de documentos de texto limpio y bien estructurado. En la realidad empresarial, los documentos tienen layouts complejos que los parsers genéricos manejan mal: contratos legales con footnotes de múltiples páginas y referencias cruzadas, reportes financieros con tablas de doble entrada y gráficos embebidos en varias columnas, manuales técnicos de aeronáutica con diagramas de subsistemas numerados, artículos científicos en formato de dos columnas con ecuaciones matemáticas, presentaciones de PowerPoint con texto en cuadros flotantes superpuestos. Cada uno de estos formatos tiene peculiaridades de extracción que un parser genérico convierte en ruido, y el ruido en la etapa de extracción es irrecoverable en la etapa de recuperación.

El problema más común y más impactante es el **PDF con layout complejo**. El formato PDF almacena los elementos de texto como objetos posicionados en el espacio 2D de la página; el orden de esos objetos en el archivo no necesariamente corresponde al orden de lectura lógico del documento. Un PDF de dos columnas extraído con `pdfplumber.extract_text()` sin configuración de layout produce texto que alterna líneas de la columna izquierda y la derecha, destruyendo la coherencia semántica del contenido: la primera oración del primer párrafo de la columna izquierda va seguida de la primera oración del primer párrafo de la columna derecha, produciendo texto incoherente que el modelo de embedding no puede representar bien.

Docling de IBM es actualmente la herramienta open source de mayor calidad para este problema: usa modelos de visión entrenados específicamente en comprensión de layouts de documentos para identificar regiones de texto, tablas, imágenes y su jerarquía espacial, y reconstruye el orden de lectura lógico correcto incluso para layouts complejos de múltiples columnas. La alternativa managed de mayor calidad es **Azure Document Intelligence**: ofrece extracción de texto con preservación de layout, detección de tablas y formularios preentrenados, y soporte para documentos escaneados con OCR de alta calidad como servicio REST sin necesidad de gestionar infraestructura de modelos de visión.

Las **tablas en PDFs** tienen tres niveles de complejidad con herramientas distintas para cada uno. Tablas con bordes explícitos (líneas dibujadas que delimitan celdas): Camelot en modo "lattice" alcanza accuracy >95% extrayendo la estructura de la tabla en formato DataFrame de pandas. Tablas sin bordes con alineación de columnas (el layout espacial del texto implica la estructura tabular): Camelot en modo "stream" alcanza accuracy del 70–85%, con errores frecuentes en tablas de muchas columnas o con valores que no tienen separación clara. Tablas distribuidas en múltiples páginas: requieren modelos de comprensión visual como Docling o Azure Document Intelligence. Las tablas extraídas correctamente se representan en Markdown con separadores `|` para indexación textual: `| Producto | Q1 | Q2 | Q3 |` es un formato que los modelos de embedding pueden representar semánticamente y que los LLMs pueden interpretar correctamente al generar respuestas.

Los **documentos técnicos con ecuaciones matemáticas** requieren herramientas especializadas. Las ecuaciones renderizadas como imágenes embebidas en el PDF son las más difíciles: solo OCR matemático especializado puede extraerlas. **NOUGAT** (Neural Optical Understanding for Academic Documents, Meta Research) convierte PDFs científicos completos a formato Markdown con ecuaciones en notación LaTeX con alta fidelidad, siendo la opción más robusta para corpus de papers científicos de física, química o matemáticas. Para documentos que mezclan texto y ecuaciones, la estrategia de identificar las páginas con alta densidad de fórmulas y procesarlas con NOUGAT mientras el resto se procesa con Docling es un balance práctico entre costo computacional y calidad.

El **corpus multilingüe** presenta el desafío de seleccionar el modelo de embedding correcto. Los modelos monolingüe tienen mejor rendimiento en un idioma específico pero no sirven para un corpus que mezcla documentos en inglés, español, francés y alemán. **BGE-M3** (BAAI) y **voyage-multilingual-2** (Voyage AI) son los modelos multilingüe de mayor calidad que alinean textos de diferentes idiomas en el mismo espacio vectorial, permitiendo que una query en español recupere documentos relevantes en inglés y viceversa. Para corpus que tienen documentos en más de 5 idiomas, la detección automática del idioma de cada documento (con la librería `langdetect` o `fasttext-language-identification`) y el registro del idioma como metadato permiten filtrar las búsquedas por idioma cuando el usuario lo requiere explícitamente.

Desafíos técnicos específicos y sus soluciones

- **PDFs con múltiples columnas:** usar Docling para detección automática de columnas con modelo de layout analysis; alternativa: pdfplumber con análisis de bounding boxes

(`extract_words()` y ordenamiento por coordenadas X/Y) para layout manual; nunca usar `extract_text()` sin configuración en PDFs de múltiples columnas.

- **Tablas en PDFs:** Camelot lattice (>95% accuracy, tablas con bordes), Camelot stream (70–85%, tablas sin bordes), Docling o Azure Document Intelligence (tablas en múltiples páginas, documentos escaneados); representar en Markdown con `|` para indexación y comprensión por el LLM.
- **Ecuaciones matemáticas:** NOUGAT (Meta) para papers científicos completos en formato LaTeX-Markdown; MathPix Snip API para ecuaciones individuales como imágenes; para corpus mixto, identificar páginas con alta densidad de fórmulas y procesarlas con herramientas especializadas.
- **Documentos técnicos con código:** chunking por función o clase (no por tamaño fijo de tokens); preservar firma, docstrings y código completo en el mismo chunk; voyage-code-3 como modelo de embedding para chunks con código (mejor representación de identificadores y APIs).
- **Corpus multilingüe:** detectar idioma con `langdetect` o `fasttext`; usar BGE-M3 o voyage-multilingual-2 para búsqueda cross-lingual; registrar el idioma como metadato para filtrado opcional por idioma en las búsquedas.
- **Documentos escaneados (PDFs de imagen):** Tesseract (open source, gratuito), AWS Textract o Azure OCR (managed, mayor accuracy); la calidad del OCR es el límite de la calidad del índice; documentos escaneados a <150 DPI producen texto con alta tasa de errores que degrada Recall@K.

Nota del Arquitecto: El inventario del corpus antes de diseñar el pipeline de ingesta es uno de los pasos más omitidos y más valiosos. Dedicar un día a analizar una muestra de 100 documentos representativos del corpus —identificando los tipos de layout, los formatos de tablas, los idiomas, la presencia de ecuaciones o código— permite priorizar las herramientas de extracción correctamente. En un corpus de reportes financieros PDF donde el 40% de la información clave está en tablas, invertir en Docling desde el inicio vale el costo; en un corpus de emails de soporte técnico en texto plano, Docling es un overhead innecesario.

El dominio real del corpus determina cuáles de estos desafíos son prioritarios y qué herramientas justifican su costo de implementación y operación. La última sección del capítulo recoge los principios de adopción de extensiones RAG que guían estas decisiones.

Módulo 6 – Capítulo 08 – Sección 06

Cierre: las extensiones de RAG amplían el dominio del problema resuelto

Las extensiones del RAG estándar que este capítulo ha presentado —multimodal, sobre datos estructurados, basado en grafos, con herramientas externas— no son variantes experimentales de dudosa utilidad en producción. Son respuestas ingenieriles a limitaciones concretas del RAG vectorial estándar que emergen cuando el sistema se enfrenta a la diversidad real del conocimiento empresarial. Una organización que solo indexa documentos de texto plano ignora el 40–60% de su conocimiento, que reside en tablas de bases de datos, imágenes de manuales, grabaciones de reuniones y presentaciones visuales. Un sistema RAG que no puede responder preguntas sobre datos actuales (precio de acción, estado de pedido, capacidad de inventario) tiene una brecha de utilidad significativa para usuarios de negocio. Estas extensiones no son features adicionales: son la diferencia entre un sistema de consulta de conocimiento limitado y un sistema de asistencia empresarial completo.

El criterio de adopción de cada extensión debe ser empírico y económico. Para cada extensión, el AI Engineer debe responder tres preguntas: ¿cuántos usuarios hacen preguntas que esta extensión resuelve, con qué frecuencia? ¿Cuál es el costo de implementar y operar esta extensión (desarrollo, infraestructura, mantenimiento)? ¿Cuál es la mejora de satisfacción del usuario (medida en las métricas de evaluación) que produce? Solo si el numerador (impacto) supera el denominador (costo) de forma clara, la extensión justifica la inversión. GraphRAG con un costo de construcción 20 veces mayor que el RAG vectorial estándar se justifica cuando el 40% de las queries del sistema son de síntesis global; no se justifica cuando el 95% de las queries son factuales específicas sobre documentos individuales.

El criterio técnico de adopción que complementa el económico es la **medición antes de la decisión**: implementar una versión mínima o un proxy de la extensión, ejecutarla sobre un subset del corpus y las queries más representativas del caso de uso, y medir el delta de calidad en el golden dataset antes de comprometerse a la implementación completa. HyDE se puede prototipar en una tarde y evaluar su impacto en el Recall@5 del dataset de evaluación; el

resultado dice si los 500ms de latencia adicional y el costo extra del LLM están justificados. Text-to-SQL se puede prototipar con el esquema de las tablas más relevantes y 10 queries de ejemplo para estimar el porcentaje de queries que genera SQL correcto antes de construir la infraestructura completa de validación y seguridad.

La **adopción incremental** es la estrategia correcta: empezar con el RAG vectorial estándar bien implementado y con evaluación correcta, identificar qué tipos de preguntas el sistema no puede responder o responde mal, y adoptar exactamente las extensiones que resuelven esos casos de uso no cubiertos. Esta estrategia contrasta con la alternativa de construir desde el inicio un sistema con todas las extensiones posibles: ese sistema tiene mayor complejidad operacional, mayor costo de infraestructura, mayor superficie de fallos, y frecuentemente no mejora la calidad de las respuestas para el 80% de las queries que el RAG estándar ya manejaba bien.

Con las extensiones del RAG cubiertas, el módulo puede retomar el camino de la madurez operacional del sistema base. El Capítulo 9 aborda la optimización sistemática del sistema RAG construido: cómo diagnosticar los tipos de errores más frecuentes, cómo optimizar el retriever y el generador de forma independiente, y cómo gestionar el costo operacional a escala.

"Todo debería hacerse tan simple como sea posible, pero no más simple." — atribuido a Albert Einstein; en ingeniería de sistemas, la complejidad adicional solo se justifica cuando resuelve problemas reales que la solución más simple no puede resolver.